

nvidia-nccl源码阅读（主要介绍流程）

📘 本页面由边俊柯于2022/06/17迁移自wiki 陈庆澍空间

一、nccl demo

</>

C++ | 收起 ^

```
1 #include <stdio.h>
2 #include "cuda_runtime.h"
3 #include "nccl.h"
4 #include "mpi.h"
5 #include <unistd.h>
6 #include <stdint.h>
7
8
9 #define MPICHECK(cmd) do { \
10     int e = cmd; \
11     if( e != MPI_SUCCESS ) { \
12         printf("Failed: MPI error %s:%d '%d'\n", \
13             __FILE__, __LINE__, e); \
14         exit(EXIT_FAILURE); \
15     } \
16 } while(0)
17
18
19 #define CUDACHECK(cmd) do { \
20     cudaError_t e = cmd; \
21     if( e != cudaSuccess ) { \
22         printf("Failed: Cuda error %s:%d '%s'\n", \
23             __FILE__, __LINE__, cudaGetErrorString(e)); \
24         exit(EXIT_FAILURE); \
25     } \
26 } while(0)
27
28
29 #define NCCLCHECK(cmd) do { \
30     ncclResult_t r = cmd; \
31     if (r!= ncclSuccess) { \
```

```
32     printf("Failed, NCCL error %s:%d '%s'\n", \
33           __FILE__, __LINE__, ncclGetErrorString(r)); \
34     exit(EXIT_FAILURE); \
35 } \
36 } while(0)
37
38
39 static uint64_t getHostHash(const char* string) {
40     // Based on DJB2, result = result * 33 + char
41     uint64_t result = 5381;
42     for (int c = 0; string[c] != '\0'; c++){
43         result = ((result << 5) + result) + string[c];
44     }
45     return result;
46 }
47
48
49 static void getHostName(char* hostname, int maxlen) {
50     gethostname(hostname, maxlen);
51     for (int i=0; i< maxlen; i++) {
52         if (hostname[i] == '.') {
53             hostname[i] = '\0';
54             return;
55         }
56     }
57 }
58
59
60 int main(int argc, char* argv[])
61 {
62     int size = 32*1024*1024;
63
64
65     int myRank, nRanks, localRank = 0;
66
67
68     //initializing MPI
69     MPICHECK(MPI_Init(&argc, &argv));
70     MPICHECK(MPI_Comm_rank(MPI_COMM_WORLD, &myRank));
71     MPICHECK(MPI_Comm_size(MPI_COMM_WORLD, &nRanks));
72
73
74     //calculating localRank based on hostname which is used in selecting a GPU
```

```
75  uint64_t hostHashs[nRanks];
76  char hostname[1024];
77  getHostName(hostname, 1024);
78  hostHashs[myRank] = getHostHash(hostname);
79  MPICHECK(MPI_Allgather(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL, hostHashs,
sizeof(uint64_t), MPI_BYTE, MPI_COMM_WORLD));
80  for (int p=0; p<nRanks; p++) {
81      if (p == myRank) break;
82      if (hostHashs[p] == hostHashs[myRank]) localRank++;
83  }
84
85
86  ncclUniqueId id;
87  ncclComm_t comm;
88  float *sendbuff, *recvbuff;
89  cudaStream_t s;
90
91
92  //get NCCL unique ID at rank 0 and broadcast it to all others
93  if (myRank == 0) ncclGetUniqueId(&id);
94  MPICHECK(MPI_Bcast((void *)&id, sizeof(id), MPI_BYTE, 0, MPI_COMM_WORLD));
95
96
97  //picking a GPU based on localRank, allocate device buffers
98  CUDACHECK(cudaSetDevice(localRank));
99  CUDACHECK(cudaMalloc(&sendbuff, size * sizeof(float)));
100 CUDACHECK(cudaMalloc(&recvbuff, size * sizeof(float)));
101 CUDACHECK(cudaStreamCreate(&s));
102
103
104 //initializing NCCL
105 NCCLCHECK(ncclCommInitRank(&comm, nRanks, id, myRank));
106
107
108 //communicating using NCCL
109 NCCLCHECK(ncclAllReduce((const void*)sendbuff, (void*)recvbuff, size,
ncclFloat, ncclSum,
110 comm, s));
111
112
113 //completing NCCL operation by synchronizing on the CUDA stream
114 CUDACHECK(cudaStreamSynchronize(s));
115
```

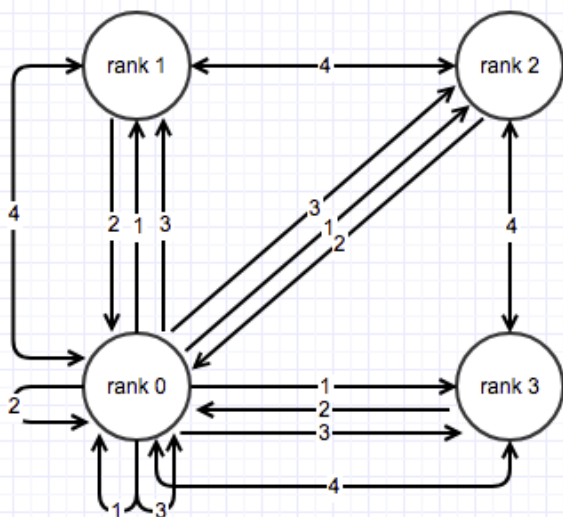
```

116
117 //free device buffers
118 CUDACHECK(cudaFree(sendbuff));
119 CUDACHECK(cudaFree(recvbuff));
120
121
122 //finalizing NCCL
123 ncclCommDestroy(comm);
124
125
126 //finalizing MPI
127 MPICHECK(MPI_Finalize());
128
129
130 printf("[MPI Rank %d] Success \n", myRank);
131 return 0;
132 }

```

二、多卡信息同步

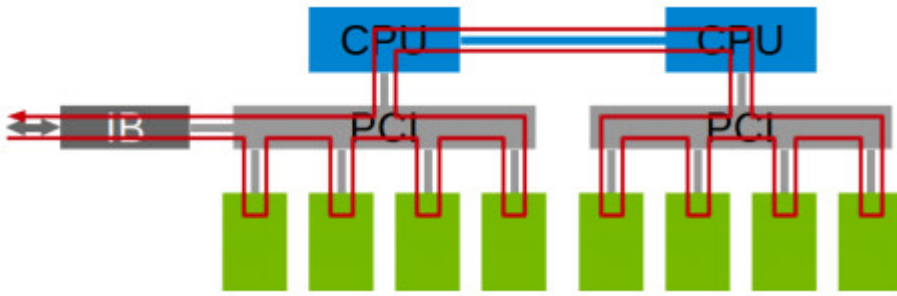
nccl进行all-reduce等collective操作时，首先需要获取总共有多少个gpu卡，每块卡的信息等元数据。为达到上述目的，用户首先需要手动给每一个卡指定一个rank id，然后，rank 0调用ncclGetUniqueld(&id)创建一个唯一id并且在rank 0上面监听在某一个端口上，该id包含了rank 0的ip和rank 0监听的端口。然后，rank 0把id发送给所有的rank（此步骤需要用户自己去实现，上述demo中通过MPI传递id）。所有rank接收到这个id后，parse这个id，根据parse结果与rank 0建立连接，然后把自己的信息发送给rank 0。rank 0根据其他rank发来的信息，与其他rank 建立连接，并且根据接收到的信息随意的确定某个rank的prev节点和next节点，其他rank从rank 0接收到自己的prev和next节点后，建立一个ring用于传递元信息（此ring仅仅用于传递元信息，后续实际数据传递的环需要根据各rank元信息来确定）。



1. rank 0 sends unique id to all ranks
2. all ranks parse id and connect to rank 0
3. rank 0 connect to all ranks and send prev&next rank to all node
4. each node connect to its prev&next rank

三、创建ring

通过章节二的介绍，此时所有rank已经连接成一个ring，可以用于交换元数据。



NCCL将网络分为三个不同的层次：P2P，shm，网络。通讯时，优先级为P2P 高于 shm 高于 网络。P2P又区分是否有NVLINK。

在创建ring的过程中，首先通过元数据的ring获取所有rank的信息（包括rank之间是否有nvlink，是否能p2p通讯，pci之间的距离，中间有几块网卡等）。ring创建过程中，能p2p有限p2p（包含nvlink），然后共享内存，最后走网络。具体如何根据所有元信息创建ring的过程和算法，没有细看。

四、nccl数据传输

[nvidia-nccl介绍](#)和[大规模分布式GPU训练](#)两篇wiki介绍了broadcast和all-reduce操作的算法，本章节主要涉及数据同步和kernel实现。nvidia-nccl在实现过程中，用一个大的kernel来实现collective操作，用来减少小kernel启动的消耗。在大kernel中，需要传递的数据被分成小的chunk，每次kernel会接收prev rank的数据，与自己chunk的数据进行处理（比如all-reduce中可能是加法），处理完毕后发送给自己的next rank。在确定自己已经完整的接收到prev rank的数据，以及自己的数据可以发送给next rank是通过共享内存来实现的。prev节点发送完数据时，会修改某bit或内存，当前rank发现某bit或内存改变时（spin等待），则可以处理数据，数据处理完之后，改某bit或者内存，则发送模块（网卡or其他）可以把数据发给next节点